Draft Manager - Draft管理UIコンポーネント

概要

目的

MAIML編集中の作業内容を一時保存（Draft）として管理するためのUIコンポーネントです。自動保存/手動保存の管理、Draft読み込み、削除、検索・ソート機能を提供します。

使用場面

- MAIML Petri Netエディタで作業中にDraftを保存する時
- 過去のDraftを読み込んで作業を再開する時
- 自動保存機能を有効/無効にする時
- 不要なDraftを削除する時

アクセス権限

- **実験管理者（Experiment Admin）**: 利用可能
- MAIML編集権限を持つユーザーのみアクセス可能
- 自分が保存したDraftのみ表示・管理可能

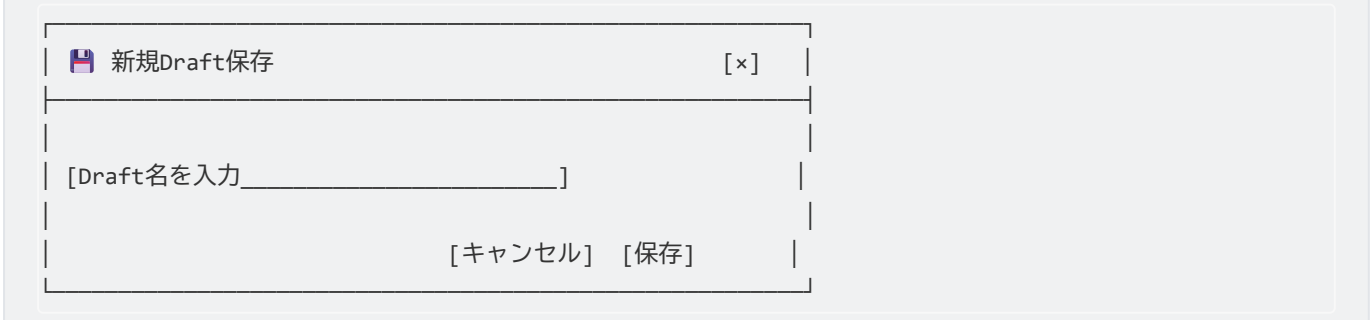
画面構成

レイアウト構造

```
┌──────────────────────────────────────────────────────┐
│ [フォルダアイコン] Draft管理                    [×]  │
├──────────────────────────────────────────────────────┤
│ [💾 新規保存] [🔄 更新] [⚪/🟢 自動保存]              │
│                          [検索...] [ソート選択]      │
│                                                      │
```



新規保存モーダル



UI要素一覧

メインモーダル

1. **モーダルヘッダー**
   - アイコン: FolderOpenOutlined
   - タイトル: "Draft管理"
   - [×]閉じるボタン

2. **ツールバー（上部）**
   - [💾 新規保存]ボタン（primary、SaveOutlined）
   - [🔄 更新]ボタン（ReloadOutlined）
   - 自動保存トグル: Switch + "🟢/○ 自動保存"ラベル
   - スペーサー（flex: 1）
   - 検索Input（prefix: SearchOutlined、幅200px、allowClear）
   - ソートSelect（幅150px）
     - 🗓️ 日付順（date）
     - 📝 名前順（name）
     - 🔢 ノード数順（nodeCount）

3. **自動保存ステータスAlert**（autoSaveEnabled && currentDraftKey の場合のみ表示）
   - メッセージ: "🟢 自動保存有効 | 編集中: {key} | 間隔: 30秒"
   - タイプ: success

4. **エラーAlert**（error存在時のみ表示）
   - タイトル: "エラー"
   - 説明: エラーメッセージ
   - タイプ: error、showIcon

5. **自動保存セクション**（autoSaveDrafts.length > 0 の場合のみ表示）
   - Card（size="small"、背景色: #f0f8ff）
   - タイトル: "🤖 自動保存（最大3世代）"（青色強調）
   - Select: 世代選択（最新/1世代前/2世代前 - MM/DD HH:MM形式）
   - Tag: 🔵 Nノード（青色）

- Tag: 🔗 Nアーク（青色）
- [読み込み]ボタン（primary、FolderOpenOutlined）

6. **Divider**（manualDrafts.length > 0 の場合のみ表示）
   - テキスト: "💾 手動保存（N件）"（強調）
7. **手動保存一覧**（List + Card）
   - Card（size="small"、marginBottom: 8px）
   - Draft名（強調）+ 保存日時（右寄せ、グレー）
   - Tag: 🔵 Nノード
   - Tag: 🔗 Nアーク
   - Tag: 📋 Nテンプレート
   - 元ファイル名（グレー、フォントサイズ12px）
   - [読み込み]ボタン（primary、small、FolderOpenOutlined）
   - [削除]ボタン（danger、small、DeleteOutlined、Popconfirm付き）
8. **空状態表示**（Draft 0件の場合）
   - テキスト: "🔭 Draftがありません"（グレー、中央揃え）
   - 検索結果0件メッセージ（searchQuery存在時）
9. **ローディング表示**（isLoading時）
   - Spin（size="large"、中央揃え、padding: 40px 0）

新規保存モーダル

1. **モーダルヘッダー**
   - タイトル: "💾 新規Draft保存"
2. **Draft名Input**
   - placeholder: "Draft名を入力"
   - Enter押下で保存実行
3. **フッター**

- [キャンセル]ボタン
- [保存]ボタン（confirmLoading付き）

機能仕様

Props定義

```
interface DraftManagerProps {
  show: boolean;                                    // モーダル表示フラグ
  onClose: () => void;                              // 閉じるコールバック
  onLoadDraft: (data: Draft & { fileName?: string }) => void;  // Draft読み込みコールバック
  currentEditState: EditState | null;               // 現在の編集状態
  userId: string;                                   // ユーザーID
  autoSaveEnabled?: boolean;                        // 自動保存有効フラグ
  onAutoSaveToggle?: (enabled: boolean) => void;    // 自動保存トグルコールバック
  currentDraftKey?: string | null;                  // 現在編集中のDraft Key
}
```

State管理

```
// useDraftManager hook の戻り値
const {
  drafts,          // Draft一覧
  isLoading,       // ローディング状態
  error,           // エラーメッセージ
  fetchDrafts,     // Draft一覧取得
  saveDraft,       // Draft保存
  deleteDraft,     // Draft削除
} = useDraftManager(currentEditState, { userId });

// ローカルState
const [searchQuery, setSearchQuery] = useState('');                          // 検索クエリ
const [showSaveDialog, setShowSaveDialog] = useState(false);                 // 保存ダイアログ表示
const [newDraftName, setNewDraftName] = useState('');                        // 新規Draft名
const [sortBy, setSortBy] = useState<'date' | 'name' | 'nodeCount'>('date');  // ソート種別
```

```
const [selectedAutoSaveGeneration, setSelectedAutoSaveGeneration] = useState(0);  // 自動保存世代選択
```

カスタムフック

useDraftManager

```
import useDraftManager from '../hooks/useDraftManager';
```

機能: Draft管理のビジネスロジックを提供

戻り値:

- `drafts` : Draft[]
- `isLoading` : boolean
- `error` : string | null
- `fetchDrafts()` : Draft一覧取得
- `saveDraft(data)` : Draft保存
- `deleteDraft(draftKey)` : Draft削除

主要機能

1. 初期化処理

userIdバリデーション

```
useEffect(() => {
  if (!userId || userId === 'default-user' || userId === 'unknown-user') {
    console.error('❌ [DraftManager V2] Invalid userId:', userId);
```

```
  }
}, [userId]);
```

動作:

- 不正なuserIdを検出してコンソールエラー出力
- デフォルト値や不明なユーザーを防止

モーダル表示時のDraft取得

```
useEffect(() => {
  if (show) {
    console.log('📄 [DraftManager V2] モーダル表示 - Draft一覧取得');
    fetchDrafts();
  }
}, [show, fetchDrafts]);
```

動作:

- モーダルが表示されたらDraft一覧を取得
- 常に最新のDraftリストを表示

2. 新規保存ダイアログ

ダイアログを開く

```
const handleOpenSaveDialog = () => {
  // ダイアログを開く際に初期値を設定
  if (currentEditState?.fileName) {
    const defaultName = `Draft - ${currentEditState.fileName} - ${new
Date().toLocaleString('ja-JP')}`;
    setNewDraftName(defaultName);
  }
  setShowSaveDialog(true);
};
```

動作:

1. 編集中のファイル名から初期Draft名を生成
2. 形式: `Draft - {fileName} - {日時}`
3. 保存ダイアログを表示

新規保存実行

```
const handleSaveNew = async () => {
  if (!newDraftName.trim()) {
    Modal.error({ title: 'エラー', content: 'Draft名を入力してください' });
    return;
  }

  try {
    const draftKey = `draft_${Date.now()}`;

    await saveDraft({
      draftKey,
      draftName: newDraftName,
      originalFileKey: currentEditState?.fileName || '',
      nodes: currentEditState?.nodes || [],
      edges: currentEditState?.edges || [],
      templates: currentEditState?.templates || [],
      documentData: currentEditState?.documentData,
      lastSavedAt : new Date().toISOString(),
    });

    Modal.success({ title: '保存完了', content: 'Draftを保存しました' });
    setShowSaveDialog(false);
    setNewDraftName('');
  } catch (err) {
    const errorMessage = err instanceof Error ? err.message : 'Unknown error';
    Modal.error({ title: '保存エラー', content: errorMessage });
  }
};
```

動作:

1. Draft名の必須チェック
2. タイムスタンプベースのdraftKeyを生成（`draft_{timestamp}`）
3. currentEditStateから必要なデータを抽出
4. saveDraft()を呼び出してサーバーに保存
5. 成功時はモーダルを閉じて入力をリセット

6. エラー時はエラーモーダル表示

3. Draft読み込み

```
const handleLoadDraft = async (draft: Draft) => {
  // キャンバス上にオブジェクトが存在する場合は警告
  const hasExistingContent =
    currentEditState &&
    ((currentEditState.nodes && currentEditState.nodes.length > 0) ||
      (currentEditState.edges && currentEditState.edges.length > 0) ||
      (currentEditState.templates && Object.keys(currentEditState.templates).length > 0));

  if (hasExistingContent) {
    const objectCounts = [
      currentEditState.nodes?.length > 0 ? `${currentEditState.nodes.length}個のノード` : '',
      currentEditState.edges?.length > 0 ? `${currentEditState.edges.length}個のアーク` : '',
      currentEditState.templates && Object.keys(currentEditState.templates).length > 0
        ? `${Object.keys(currentEditState.templates).length}個のテンプレート`
        : '',
    ]
      .filter(Boolean)
      .join('、');

    Modal.confirm({
      title: '編集中のMaiMLを破棄して読み込みますか？',
      content: `現在のキャンバスには以下が存在します：\n${objectCounts}\n\n現在の編集内容は失われます。`,
      onOk: async () => {
        // Draft読み込み実行
        await loadDraftData(draft);
      },
    });
  } else {
    // コンテンツがない場合はそのまま読み込み
    await loadDraftData(draft);
  }
};

// 実際の読み込み処理（共通化）
const loadDraftData = async (draft: Draft) => {
  try {
    const data = await draftService.fetchDraftById(draft.draftKey);

    const loadData = {
      ...data,
      fileName: data.originalFileKey || data.draftName,
    };
```

```
    onLoadDraft(loadData);
    onClose();
  } catch (err) {
    const errorMessage = err instanceof Error ? err.message : 'Unknown error';
    Modal.error({ title: '読み込みエラー', content: errorMessage });
  }
};
```

動作フロー:

1. 現在のキャンバスに編集内容があるかチェック
2. 編集内容がある場合:
   - オブジェクト数をカウント（ノード/アーク/テンプレート）
   - 確認ダイアログを表示
   - OKクリック → Draft読み込み実行
3. 編集内容がない場合:
   - 確認なしで即座に読み込み
4. fetchDraftById()でDraft詳細を取得
5. fileNameを付与して親コンポーネントに渡す
6. モーダルを閉じる

4. Draft削除

```
const handleDelete = async (draftKey: string, draftName: string) => {
  try {
    await deleteDraft(draftKey);
    Modal.success({ title: '削除完了', content: `"${draftName}" を削除しました` });
  } catch (err) {
    const errorMessage = err instanceof Error ? err.message : 'Unknown error';
    Modal.error({ title: '削除エラー', content: errorMessage });
  }
};
```

動作:

1. Popconfirmで削除確認（UI側）
2. deleteDraft()でサーバーから削除
3. 成功時は成功メッセージ表示
4. fetchDrafts()が自動的に一覧を更新（useDraftManager内部）

5. 自動保存トグル

```
const handleAutoSaveToggle = (checked: boolean) => {
  if (onAutoSaveToggle) {
    onAutoSaveToggle(checked);
  }
};
```

動作:

- Switch切り替え → 親コンポーネントのonAutoSaveToggleコールバック呼び出し
- 親側で自動保存タイマーの開始/停止を制御

6. フィルタリング・ソート（useMemo）

```
const { autoSaveDrafts, manualDrafts } = useMemo(() => {
  // drafts が配列であることを保証
  const draftArray = Array.isArray(drafts) ? drafts : [];

  let filtered = draftArray;

  // 🔍 検索フィルター
  if (searchQuery.trim()) {
    const query = searchQuery.toLowerCase();
    filtered = draftArray.filter(
      (draft) =>
        draft.draftName.toLowerCase().includes(query) ||
        draft.originalFileKey?.toLowerCase().includes(query)
    );
  }

  // 🤖 自動保存と手動保存に分ける
  const autoSaves = filtered
    .filter((d : Draft) => d.draftName && d.draftName.includes('自動保存'))
```

```
    .sort((a, b) => new Date(b.lastSavedAt).getTime() - new Date(a.lastSavedAt).getTime());

  const manuals = filtered.filter((d) => !d.draftName || !d.draftName.includes('自動保存'));

  // 💾 手動保存をソート
  const sortedManuals = [...manuals].sort((a, b) => {
    switch (sortBy) {
      case 'name':
        return a.draftName.localeCompare(b.draftName);
      case 'nodeCount':
        return ((b.nodes?.length || 0) - (a.nodes?.length || 0));
      case 'date':
      default:
        return new Date(b.lastSavedAt || new Date()).getTime() -
               new Date(a.lastSavedAt || new Date()).getTime();
    }
  });

  return {
    autoSaveDrafts: autoSaves,
    manualDrafts: sortedManuals,
  };
}, [drafts, searchQuery, sortBy]);
```

動作:

1. draftsが配列であることを保証（エラー防止）
2. 検索クエリでフィルタリング（Draft名 or 元ファイル名）
3. 自動保存と手動保存を分離（Draft名に"自動保存"が含まれるか）
4. 自動保存: 日付降順（最新が先頭）
5. 手動保存: sortByに応じてソート
   - `date`: 日付降順（デフォルト）
   - `name`: 名前昇順
   - `nodeCount`: ノード数降順

データ型定義

Draft

```
interface Draft {
  draftKey: string;                    // Draft識別キー
  draftName: string;                   // Draft名
  originalFileKey?: string;            // 元ファイル名
  nodes: any[];                        // ノード一覧
  edges: any[];                        // アーク一覧
  templates: Record<string, any>;      // テンプレート一覧
  documentData?: any;                  // ドキュメントデータ
  lastSavedAt: string;                 // 最終保存日時（ISO形式）
}
```

EditState

```
interface EditState {
  fileName?: string;                   // 編集中のファイル名
  nodes?: any[];                       // ノード一覧
  edges?: any[];                       // アーク一覧
  templates?: Record<string, any>;     // テンプレート一覧
  documentData?: any;                  // ドキュメントデータ
}
```

API連携

draftService

```
import * as draftService from '@/services/draftService';
```

fetchDraftById()

```
const data = await draftService.fetchDraftById(draft.draftKey);
```

機能: 指定されたdraftKeyのDraft詳細を取得

パラメータ:

- `draftKey` : string

戻り値: Draft

使用箇所: Draft読み込み時

関連コンポーネント

カスタムフック

useDraftManager

- **ファイルパス**: `frontend/src/components/editorv2/hooks/useDraftManager.ts`
- **用途**: Draft管理のビジネスロジックを提供
- **機能:**
  - Draft一覧取得（fetchDrafts）
  - Draft保存（saveDraft）
  - Draft削除（deleteDraft）
  - ローディング状態管理
  - エラー管理

親コンポーネント

PetriNetEditorScreen / PetriNetEditorInner

- **ファイルパス**: `frontend/src/components/editorv2/PetriNetEditorScreen.tsx`
- **用途**: MAIML Petri Netエディタ画面
- **連携:**

- DraftManagerを表示/非表示
- onLoadDraftでDraftデータを受け取って復元
- autoSaveEnabledフラグで自動保存を制御
- currentEditStateで現在の編集状態を渡す

使用コンポーネント（Ant Design）

- **Modal**: モーダルダイアログ
- **Button**: ボタン
- **Input**: 検索入力
- **List**: Draft一覧
- **Card**: Draftカード
- **Space**: 要素間スペース
- **Spin**: ローディング表示
- **Alert**: ステータス/エラー表示
- **Select**: ソート選択
- **Switch**: 自動保存トグル
- **Tag**: ノード数/アーク数表示
- **Divider**: セクション区切り
- **Popconfirm**: 削除確認

エラーハンドリング

Draft名必須チェック

```
if (!newDraftName.trim()) {
  Modal.error({ title: 'エラー', content: 'Draft名を入力してください' });
  return;
}
```

API通信エラー

```
try {
  await saveDraft({ ... });
  Modal.success({ title: '保存完了', content: 'Draftを保存しました' });
} catch (err) {
  const errorMessage = err instanceof Error ? err.message : 'Unknown error';
  Modal.error({ title: '保存エラー', content: errorMessage });
}
```

編集内容破棄の確認

```
Modal.confirm({
  title: '編集中のMaiMLを破棄して読み込みますか？',
  content: `現在のキャンバスには以下が存在します：\n${objectCounts}\n\n現在の編集内容は失われます。
`,
  onOk: async () => {
    // Draft読み込み実行
  },
});
```

userIdバリデーション

```
if (!userId || userId === 'default-user' || userId === 'unknown-user') {
  console.error('❌ [DraftManager V2] Invalid userId:', userId);
}
```

パフォーマンス最適化

useMemo使用

```
const { autoSaveDrafts, manualDrafts } = useMemo(() => {
  // フィルタリング・ソート処理
}, [drafts, searchQuery, sortBy]);
```

メリット:

- drafts/searchQuery/sortByが変更された場合のみ再計算
- 不要な再レンダリングを防止

配列チェック

```
const draftArray = Array.isArray(drafts) ? drafts : [];
```

メリット:

- draftsが配列でない場合のエラーを防止
- 型安全性の向上

使用例

PetriNetEditorInner内での使用

```
const [showDraftManager, setShowDraftManager] = useState(false);
const [autoSaveEnabled, setAutoSaveEnabled] = useState(false);
const [currentDraftKey, setCurrentDraftKey] = useState<string | null>(null);

// Draft読み込みハンドラー
const handleLoadDraft = (data: Draft & { fileName?: string }) => {
  // Draftデータをエディタに復元
  setNodes(data.nodes || []);
  setEdges(data.edges || []);
  setTemplates(data.templates || {});
  setDocumentData(data.documentData);
  setFileName(data.fileName || data.draftName);
```

```
  console.log('✅ Draft読み込み完了');
};

// 自動保存トグル
const handleAutoSaveToggle = (enabled: boolean) => {
  setAutoSaveEnabled(enabled);

  if (enabled) {
    // 自動保存タイマー開始
    startAutoSave();
  } else {
    // 自動保存タイマー停止
    stopAutoSave();
  }
};

// DraftManager表示
<DraftManager
  show={showDraftManager}
  onClose={() => setShowDraftManager(false)}
  onLoadDraft={handleLoadDraft}
  currentEditState={{
    fileName: fileName,
    nodes: nodes,
    edges: edges,
    templates: templates,
    documentData: documentData,
  }}
  userId={userId}
  autoSaveEnabled={autoSaveEnabled}
  onAutoSaveToggle={handleAutoSaveToggle}
  currentDraftKey={currentDraftKey}
/>
```

特記事項

自動保存の仕組み

1. **保存間隔**: 30秒（親コンポーネントのタイマーで制御）
2. **世代管理**: 最大3世代まで保持
3. **Draft名形式**: Draft名に"自動保存"を含める
4. **表示**: 自動保存セクションに分離して表示（青色背景）

Draft Key生成

```
const draftKey = `draft_${Date.now()}`;
```

- タイムスタンプベース（ミリ秒）
- 一意性を保証
- 形式: `draft_1234567890123`

fileName復元

```
const loadData = {
  ...data,
  fileName: data.originalFileKey || data.draftName,
};
```

- originalFileKeyを優先
- なければdraftNameをfileNameとして使用
- エディタタイトルバーに表示される

変更履歴

日付	バージョン	変更内容	担当者
2026-03-12	1.0.0	実装ベースでUI仕様書を新規作成	Cline
2026-XX-XX	-	V2版（Ant Design実装）完了（実装）	-
2025-XX-XX	-	Draft管理UIコンポーネント初版作成（実装）	-

実装ファイル情報

- **ファイルパス**: `frontend/src/components/editorv2/components/DraftManager.tsx`
- **コンポーネント名**: `DraftManager`
- **実装バージョン**: 2.0.0（Ant Design）
- **実装日**: 2026-XX-XX